

DOCUMENTATION

Raytracer

Participants:

Luis Fernandes
Alexis Salaun
Julien Marcq
Nicolas Nguyen

Sommaire

Introduction	
.....	3
Plan	
.....	4
Exécution du projet	
.....	10
Fichier de configuration	
.....	11

Introduction

Le Raytracer vise à concevoir et à développer un logiciel capable de générer des images en simulant le comportement de la lumière dans une scène tridimensionnelle.

Nous devons mettre en œuvre des techniques de rendu telles que le lancer de rayons pour calculer la couleur et l'éclairage de chaque pixel de l'image finale.

Cela permet d'explorer des concepts avancés de graphisme et de rendre des scènes réalistes en utilisant des algorithmes sophistiqués.

Plan (architecture du projet)

Engine :

L'Engine est un dossier qui contient le cœur de notre application de raytracing. Il orchestre l'ensemble du processus de rendu en coordonnant les différentes étapes, telles que la lecture de la scène, le calcul des interactions de la lumière, le rendu des images finales, etc.

- **Renderer :**

Le Renderer est responsable de calculer l'apparence des objets dans la scène en simulant la propagation des rayons de lumière.

Le Renderer utilise les données de la scène, comme la géométrie des objets, les matériaux, les lumières, etc., pour générer des images réalistes.

- **Scene :**

La Scène représente l'environnement virtuel qu'on souhaite rendre. Elle contient des informations sur les objets présents dans la scène, comme leur géométrie, leurs matériaux, leurs positions, leurs orientations, ainsi que les sources de lumière et les caméras.

- **Camera :**

La Caméra simule la perspective à partir de laquelle la scène est vue. Elle détermine quels rayons de lumière traversent l'objectif de la caméra et quelle partie de la scène est visible dans l'image finale. La caméra peut être configurée avec différents paramètres, tels que la position, l'orientation, l'angle de vue, la profondeur de champ, etc.

- **Exporter :**

L'Exporter (exportateur) est responsable de sauvegarder l'image rendue dans un format spécifique, ici le .ppm.

Lights :

Le dossier Lights est responsable de la gestion de la lumière dans notre moteur de rendu.

- **Ambient :**

Cette classe gère la lumière ambiante dans la scène. La lumière ambiante est une lumière naturelle provenant de toutes les directions qui permet d'éclairer les zones sombres d'une scène mal éclairée. Elle contribue à créer une atmosphère générale dans la scène et à éviter les ombres complètement noires.

- **Directional :**

Cette classe représente une lumière directionnelle similaire à la lumière du soleil. Sa position n'a pas d'influence sur l'angle d'incidence de la lumière ; seule sa direction compte. Elle émet une lumière parallèle dans une direction donnée et est souvent utilisée pour simuler la lumière du soleil dans une scène.

- **Environment :**

Cette classe gère une texture 3D fournie par un fichier PPM (Portable Pixel Map). Cette texture est utilisée pour fournir un contexte lumineux à la scène, ce qui la rend plus réaliste. Elle peut représenter des environnements comme le ciel, les nuages, ou d'autres sources de lumière ambiante.

- **LightPoint :**

Cette classe représente la source de lumière la plus simple, un point de lumière ponctuel. Elle permet de créer des éclairages ponctuels dans la scène. Les sources de lumière ponctuelles émettent de la lumière dans toutes les directions à partir d'un point donné et sont souvent utilisées pour simuler des lampes ou des bougies dans une scène.

- **Spot :**

Cette classe représente une lumière dirigée en forme de cône, similaire à un projecteur. Le spot permet de créer un éclairage précis et contrôlé dans la scène. Il prend en paramètre un angle pour définir la taille du cône d'éclairage et une intensité lumineuse pour contrôler la luminosité du spot. Cette lumière est particulièrement utile pour créer des jeux de contrastes et mettre en valeur des éléments spécifiques dans la scène.

Objects :

Les objets sont l'une des principales parties du Raytracer. À l'aide d'une Factory, les objets sont chargés à partir du fichier de configuration. Le fonctionnement des objets est basé sur un système d'héritage.

- **AObject :**
Cette classe sert de classe de base pour tous les objets de votre scène. Elle initialise les attributs communs à tous les objets, tels que la taille ou les propriétés de réflexion de la lumière.
- **Plane :**
Un plan infini dans l'espace tridimensionnel. Il est défini par une équation de plan et est souvent utilisé pour représenter des surfaces planes comme des murs, des sols ou des plafonds.
- **Rect :**
Un plan avec des bords définis par des coordonnées. Il peut être utilisé pour modéliser des surfaces rectangulaires dans la scène.
- **Circle :**
Un plan avec une découpe circulaire. Utile pour représenter des objets comme des disques ou des cercles dans la scène.
- **Triangle :**
Un plan avec une découpe en forme de triangle. Il est utilisé pour modéliser des surfaces triangulaires dans la scène.
- **Cone :**
Un cône tridimensionnel défini par sa hauteur et son rayon à la base. Il peut être utilisé pour représenter des objets coniques dans la scène.
- **Cube :**
Un objet composé de plusieurs Rect assemblés pour former une forme cubique.
- **Cylinder :**
Un cylindre tridimensionnel défini par sa hauteur et son rayon. Il peut être utilisé pour modéliser des objets cylindriques comme des colonnes ou des tuyaux.

- Sphere :
Une sphère tridimensionnelle définie par son centre et son rayon.

Parser :

Le Parser est le point d'entrée pour l'ensemble du système de parsing.

- Parser :
Centralise toutes les informations récupérées pour toutes les sections du fichier de configuration pour la scène.
- RecupCamera :
Ce composant est dédié à la récupération des informations relatives à la caméra dans le fichier de configuration. Il analyse ces informations et les stocke dans une structure de données pour une utilisation dans le rendu final.
- RecupEffect :
Ce composant est dédié à la récupération des informations relatives aux effets, tels que les effets visuels ou les filtres, à partir du fichier de configuration. Il analyse ces informations et les stocke dans une structure de données pour une utilisation dans le rendu final.
- RecupLight :
Ce composant est chargé de récupérer les informations relatives aux lumières présentes dans la scène à partir du fichier de configuration. Il analyse ces informations et les stocke dans une structure de données appropriée pour une utilisation dans le moteur de rendu.
- RecupPrimitive :
Ce composant est responsable de la récupération des informations relatives aux primitives géométriques (objets) de la scène, tels que les sphères, les plans, etc., à partir du fichier de configuration. Il analyse ces informations et les stocke dans une structure de données pour une utilisation dans le moteur de rendu.
- StringUtils :
Ce fichier ou module contient des fonctions utilitaires pour manipuler les chaînes de caractères et effectuer des opérations de recherche ou de traitement dans les fichiers de configuration ou les données analysées.

Physics :

Le dossier Physics comprend les composants essentiels pour la simulation des aspects physiques dans notre moteur de rendu.

- **Color :**

Il s'agit d'un vecteur colorimétrique à 4 composantes (rouge, vert, bleu, transparence) utilisé pour représenter les couleurs dans notre rendu. Ce vecteur permet de gérer les calculs entre les rayons lumineux et les interactions avec les matériaux pour obtenir des résultats visuels réalistes.

- **Light :**

Définit une interface pour les lumières dans la scène.

Elle décrit le comportement de base des sources lumineuses, telles que les lumières directionnelles, les lumières ponctuelles ou les lumières de zone.

Cette interface peut être implémentée par différents types de lumières pour simuler divers effets d'éclairage dans la scène.

- **Point :**

Une classe qui représente un point dans l'espace tridimensionnel. Les coordonnées de ce point sont utilisées pour définir la position des objets dans la scène. Cette classe peut être utilisée pour stocker les positions des lumières, des objets, des caméras, etc.

- **Ray :**

Il s'agit d'une combinaison d'un vecteur et d'une couleur, utilisée pour représenter un rayon lumineux dans notre simulation. Ce rayon simplifie la gestion des trajectoires de lumière et est utilisé pour calculer les interactions entre la lumière et la géométrie de la scène.

- **Vector :**

Cette classe représente un vecteur tridimensionnel et est largement utilisée dans les calculs de rayons et d'éclairage. Permet de calculer les réflexions, les réfractions, les ombres, etc.

Effects :

Le dossier Effects contient différents effets visuels qui peuvent être appliqués aux images rendues pour améliorer leur apparence ou ajouter des effets artistiques.

- **Bloom :**

Le bloom est un effet visuel qui simule la diffusion de la lumière dans une image. Il crée une aura lumineuse autour des zones les plus lumineuses de l'image, ce qui donne un effet de luminosité et de douceur.

- **Aberration chromatique :**

L'aberration chromatique est un effet optique qui se produit lorsque les différentes longueurs d'onde de la lumière sont déviées de manière différente lorsqu'elles traversent un matériau. Cela se traduit généralement par un effet de déformation des couleurs autour des bords des objets dans une image.

- **GaussianBlur :**

Le flou gaussien est une technique de floutage utilisée pour adoucir les détails d'une image. Il fonctionne en appliquant un noyau de convolution gaussien à chaque pixel de l'image, ce qui produit un effet de flou progressif.

- **Clamp :**

Le clamp est une opération qui limite les valeurs d'une image à une plage spécifique. Cela signifie que les valeurs des pixels qui sont en dehors de cette plage sont ajustées pour tomber dans la plage spécifiée. Le clamp peut être utilisé pour corriger les valeurs de luminosité ou de couleur qui dépassent des limites.

Exécution du projet

Pour exécuter le programme, il faut d'abord compiler celui-ci avec le makefile

Pour lancer la compilation :

```
~/B-OOP-400> make
```

Puis lancer le programme avec :

```
~/B-OOP-400> ./raytracer <file.cfg>
```

Pour exécuter avec la sfml (interface graphique) :

```
~/B-OOP-400> ./raytracer -sfml <file.cfg>
```

<file.cfg> est le fichier de configuration pour le projets, les détails sont dans la rubrique du dessous

Fichier de configuration

Voici un exemple de fichier de configuration :

```
# Configuration of the camera
camera :
{
    resolution = { width = 1920 height = 1080 }
    position = { x = 0 y = 0 z = 0 }
    rotation = { x = 0 y = 0 z = 0 }
    direction = { x = 1 y = 0 z = 0 }
    fieldOfView = 45.0
};

# Primitives in the scene
primitives :
{
    # List of spheres
    sphere = { x = 5 y = -1.8 z = 0 radius = 1 color = { r = 0 g = 127 b = 255 } }
    sphere = { x = 5 y = 1.8 z = 0 radius = 1 color = { r = 255 g = 0 b = 0 } }
    sphere = { x = 5 y = 0 z = 0 radius = 1 color = { r = 255 g = 255 b = 255 } }

    # List of planes
    plane = { x = 1 y = 1 z = -1 normal_x = 0 normal_y = 0 normal_z = 1 color = { r = 225 g = 255 b = 255 } }

    # List of rectangles
    rectangle = { x = 9 y = -5 z = -1 size_x = 1 size_y = 1 size_z = 0 color = { r = 55 g = 55 b = 255 } }
};

# Light configuration
lights :
{
    ambient = { r = 255 g = 255 b = 255 intensity = 0.1 }
    diffuse = 0.6 # Multiplier of diffuse light
    environment = { path = assets/texture.ppm intensity = 0.5 }

    # list of point lights
    point = { x = 5 y = 0 z = 9 intensity = 5000 }
    point = { x = 5 y = -4 z = 7 intensity = 1000 }

    # list of directional lights
    directional = { x = 0 y = -1 z = -1 intensity = 0.5 }
};

# effects configuration
effects :
{
    bloom = { threshold = 0.8 sigma = 100 radius = 50 }
    gaussianBlur = { sigma = 1 radius = 8 }
};
```

Voici les points importants :

- Les commentaires sont représentés par des #
- Chaque partie sont séparées par des titres de sections (par ex: camera) et délimitées par des crochets ({ });)

Présentation des différentes parties du fichier de configuration

Nous allons vous expliquer le parsing attendu pour que le projet puisse fonctionner.

Peu importe l'ordre des sections : vous pouvez mettre la camera avant les primitives, ou l'inverse pareil pour toutes les autres sections.

Il n'est pas utile de mettre tous les paramètres de chaque section, des paramètres par défaut sont appliqués. Ici on vous présente tout ce qu'il est possible d'utiliser.

Certains paramètres peuvent être mis plusieurs fois, par exemple dans les primitives pour pouvoir mettre plusieurs objets, quand ce sera le cas, dans l'image d'exemple il sera écrit en commentaire # list of

Section camera:

Permet de régler les paramètre de la caméra

```
# Configuration of the camera
camera :
{
    resolution = { width = 1920 height = 1080 }
    position = { x = 0 y = 0 z = 0 }
    rotation = { x = 0 y = 0 z = 0 }
    direction = { x = 1 y = 0 z = 0 }
    fieldOfView = 65.0
};
```

resolution: dans l'ordre, correspond à la largeur et la hauteur de la fenêtre.

Valeur strictement supérieure 0.

Par défaut: width = 1920, height = 1080

position: La position dans le plan 3d.

Par défaut: x = 0, y = 0, z = 0

rotation: La rotation dans le plan 3d.

Par défaut: x = 0, y = 0, z = 0

direction: la direction vers laquelle la caméra regarde.

Par défaut: x = 1, y = 0, z = 0

fieldOfView: angle de vue de la caméra. Valeur entre 0 et 180.

Par défaut: 90

Section primitives:

Permet de rajouter des objets dans la scène

```
# Primitives in the scene
primitives :
{
  # List of spheres
  sphere = { x = 5 y = -1.8 z = 0 radius = 1 color = { r = 0 g = 127 b = 255 } transparency = 0 }
  sphere = { x = 5 y = 1.8 z = 0 radius = 1 color = { r = 255 g = 0 b = 0 } transparency = 0 }
  sphere = { x = 5 y = 0 z = 0 radius = 1 color = { r = 255 g = 255 b = 255 } transparency = 0 }

  # List of planes
  # plane = { x = 1 y = 1 z = -1 normal_x = 0 normal_y = 0 normal_z = 1 color = { r = 225 g = 255 b = 255 } transparency = 0 }

  # List of rectangles
  # rectangle = { x = 9 y = -5 z = -1 size_x = 1 size_y = 1 size_z = 0 color = { r = 55 g = 55 b = 255 } transparency = 0 }

  # List of cylinders
  # cylinder = { x = 5 y = 0 z = 0.5 normal_x = 0 normal_y = 0 normal_z = 1 radius = 0.5 height = 2 color = { r = 0 g = 0 b = 255 } transparency = 0 }
};
```

Pour définir plusieurs primitives, il faut juste les mettre à la suite, par exemple dans l'image ci-dessus, il y aura 3 sphères d'afficher, un plan, un rectangle et un cylindre.

spheres:

- **x, y, z** : Position dans l'espace.
Par défaut : $x = 0, y = 0, z = 0$
- **radius** : Le diamètre de la sphère, entre 0 et 1.
Par défaut : 0,5
- **r, g, b** : La couleur (rouge, vert, bleu), valeur comprise entre 0 et 255.
Par défaut : $r = 255, g = 255, b = 255$
- **transparency**: la transparence de l'objet
Par défaut : 0

planes:

- **x, y, z** : Position dans l'espace.
Par défaut : $x = 0, y = 0, z = 0$
- **normal_x, normal_y, normal_z** : La normal d'un vecteur 3d (x, y, z), représente un vecteur orthogonal à la surface à ce point, indiquant la direction perpendiculaire à celle-ci.
Par défaut : $\text{normal_x} = 0, \text{normal_y} = 0, \text{normal_z} = 0$
- **r, g, b** : La couleur r, g, b (rouge, vert, bleu), valeur comprise entre 0 et 255.

Par défaut : $r = 255, g = 255, b = 255$

- **transparency**: la transparence de l'objet

Par défaut : 0

rectangles:

- **x, y, z** : Position dans l'espace, x, y, z.

Par défaut : $x = 0, y = 0, z = 0$

- **size_x, size_y, size_z** : La taille du rectangle en 3d, en x, y, z.

Par défaut : $size_x = 1, size_y = 1, size_z = 1$

- **r, g, b** : La couleur r, g, b (rouge, vert, bleu), valeur comprise entre 0 et 255.

Par défaut : $r = 255, g = 255, b = 255$

- **transparency**: la transparence de l'objet

Par défaut : 0

cylinder:

- **x, y, z** : Position dans l'espace.

Par défaut : $x = 0, y = 0, z = 0$

- **normal_x, normal_y, normal_z** : La normal d'un vecteur 3d (x, y, z), représente un vecteur orthogonal à la surface à ce point, indiquant la direction perpendiculaire à celle-ci.

Par défaut : $normal_x = 0, normal_y = 0, normal_z = 0$

- **radius** : Le diamètre du cylindre, entre 0 et 1.

Par défaut : 0,5

- **height** : la hauteur du cylindre, valeur strictement supérieure à 0.

Par défaut : 1

- **r, g, b** : La couleur (rouge, vert, bleu), valeur comprise entre 0 et 255.

Par défaut : $r = 255, g = 255, b = 255$

- **transparency**: la transparence de l'objet

Par défaut : 0

Section lights:

Permet de paramétrer la gestion de la lumière

```
# Light configuration
lights :
{
    environment = { path = assets/texture.ppm intensity = 1 }
    # ambient = { r = 255 g = 255 b = 255 intensity = 0.1 }
    # diffuse = 0.6 # Multiplier of diffuse light
    # point = { x = 5 y = 0 z = 9 intensity = 5000 }
    # point = { x = 5 y = -4 z = 7 intensity = 1000 }
    # directional = { x = 0 y = -1 z = -1 intensity = 0.5 }
};
```

environment:

- **path** : chemin vers le fichier d'environnement, fichier en .ppm.
Par défaut : aucun fichier
- **intensity** : La valeur de la transparence de l'environnement, valeur comprise entre 0 et 1.
Par défaut : intensity = 1

ambient:

- **r, g, b** : La couleur de la lumière
Par défaut : r = 255, g = 255, b = 255
- **intensity** : La valeur de la lumière ambiante, valeur comprise entre 0 et 1.
Par défaut : intensity = 0,5

diffuse: La valeur de la lumière diffuse, valeur comprise entre 0 et 1.

Par défaut : 0.5

point:

- **x, y, z** : position du point de lumière dans l'espace 3d.
Par défaut : x = 0, y = 0, z = 0
- **intensity** : son intensité lumineuse, valeur strictement supérieure à 0.
Par défaut : intensity = 50

directional:

- **x, y, z** : direction vers lequel la lumière éclaire

Par défaut : $x = 1, y = 1, z = 1$

- **intensity** : son intensité lumineuse, valeur strictement supérieure à 0.

Par défaut : intensity = 50

Exemple d'environnement:



Section effets:

Les effets permettent d'ajouter des filtres sur l'image pour un meilleur rendu. Pour ne pas les activer, il suffit de ne pas mettre la ligne ou de la commenter

```
# effects configuration
effects :
{
    bloom = { threshold = 0.8 sigma = 100 radius = 50 }
    # gaussianBlur = { sigma = 1 radius = 8 }
};
```

bloom: Permet de simuler une lumière intense.

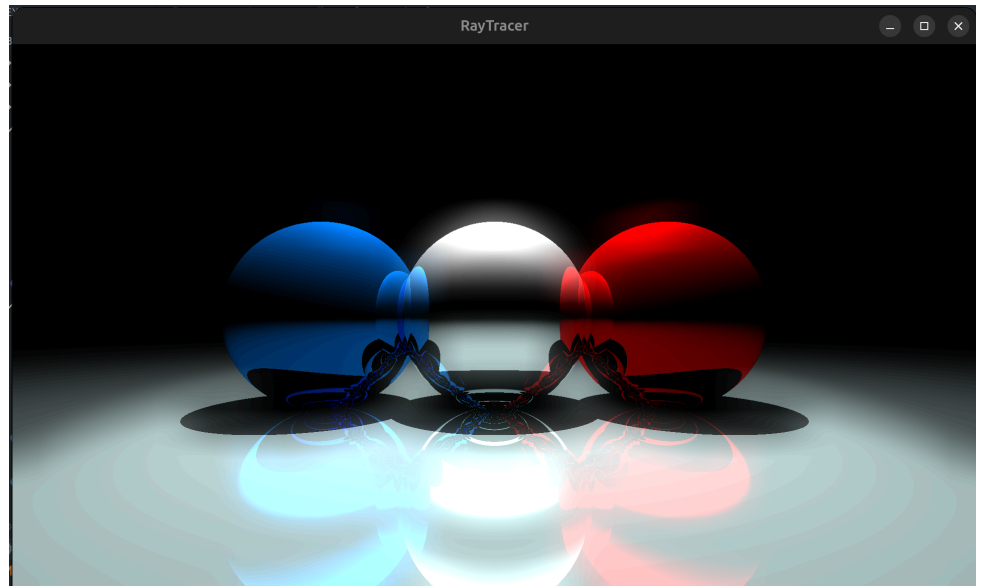
- **threshold** : seuil de déclenchement, valeur entre 0 et 1.
Par défaut : 0.8
- **sigma** : puissance de la répartition, valeur strictement supérieure à 0.
Par défaut : 100
- **radius** : étendue de l'effet, valeur strictement supérieure à 0.
Par défaut : 50

gaussianBlur: Permet de simuler du flou.

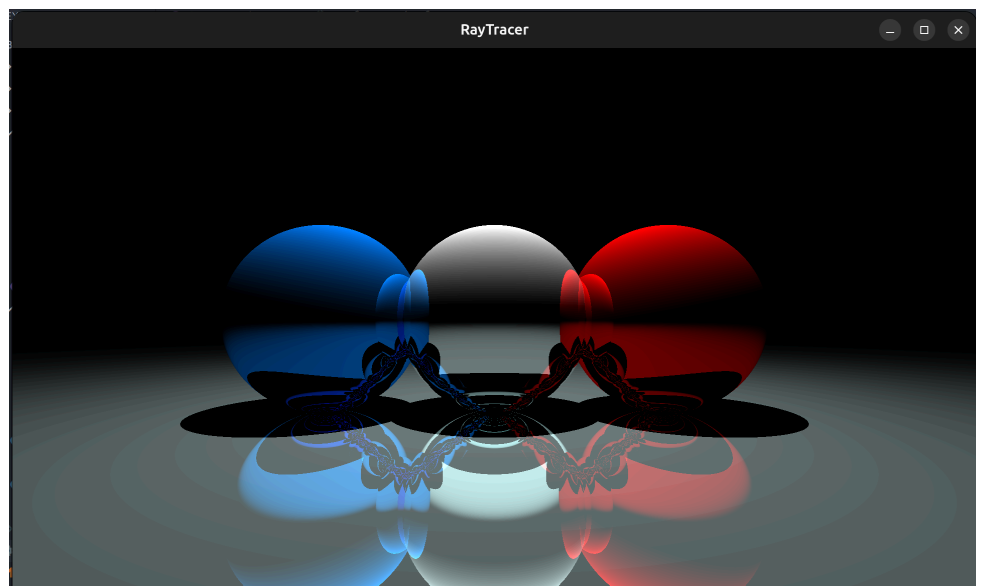
- **sigma** : puissance de l'effet, valeur entre 0 et 1.
Par défaut : 1
- **radius** : étendue de l'effet, valeur strictement supérieure à 0.
Par défaut : 0

Exemples d'effets:

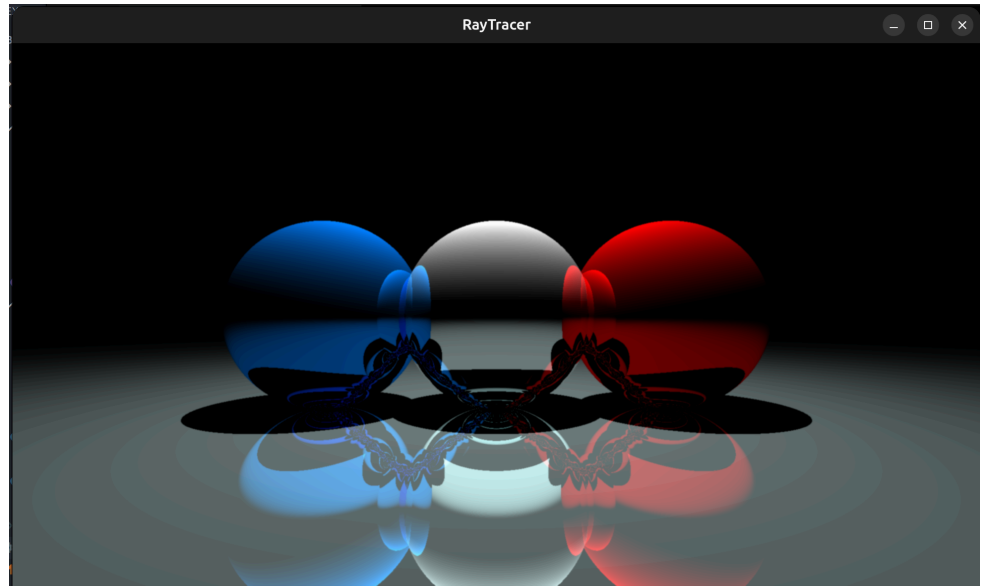
- Avec **bloom**:



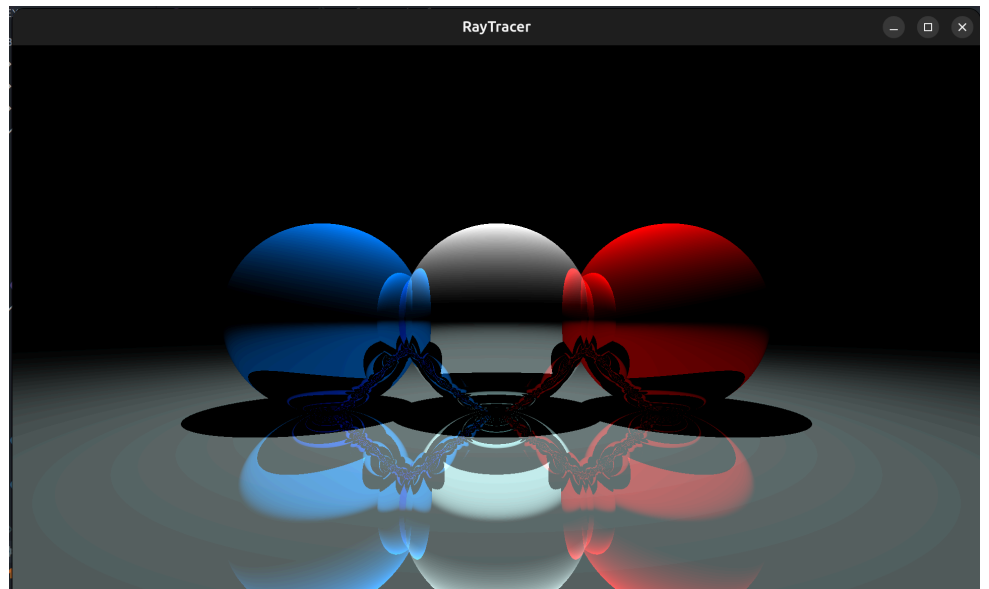
- sans **bloom**:



- Avec **gaussianBlur**:



- Sans **gaussianBlur** :



Nous contacter

Si vous voulez plus d'informations, veuillez nous contacter par mail à:

Alexis Salaun: alexis.salaun@epitech.eu

Julien Marcq: julien.marcq@epitech.eu

Luis Fernandes: luis.fernandes@epitech.eu

Nicolas Nguyen: nicolas1.nguyen@epitech.eu